AFFILA · ELJURI VOICE SIMULATOR

CONFIDENCIAL

Reporte de Ciberseguridad

ELjuri Voice Simulator

FECHA	ALCANCE	VULNERABILIDADES	ESTADO
12 de marzo de 2026	Backend & Frontend	15 identificadas	Sin resolver

Reporte de Ciberseguridad

ELjuri Voice Simulator

Fecha: 12 de marzo de 2026

Clasificación: Confidencial — Uso interno

Alcance: Código fuente en `/backend/src` y `/frontend` (excluye node_modules)

Metodología: Revisión estática de código y análisis manual

Resumen Ejecutivo

Se realizó una auditoría de seguridad completa sobre el proyecto **ELjuri Voice Simulator**, un simulador de voz basado en IA que integra Google Gemini Live, WebSockets y una interfaz Next.js. El análisis identificó **15 vulnerabilidades activas** distribuidas en 2 niveles críticos, 6 altos y 7 medios.

> **CONCLUSIÓN:** El proyecto **NO DEBE DESPLEGARSE EN PRODUCCIÓN** en su estado actual. Se requiere resolver como mínimo los ítems CRÍTICOS y ALTOS antes de cualquier despliegue.

Severidad	Cantidad	Estado
CRÍTICO	2	Sin resolver
ALTO	6	Sin resolver
MEDIO	7	Sin resolver
TOTAL	**15**	

---

1. Vulnerabilidades CRÍTICAS

CRÍTICO-01: API Key de Google expuesta en control de versiones

Campo	Detalle
Severidad	CRÍTICO
Categoría	Exposición de secretos
Archivo	`backend/.env`
Artículo LOPDP relacionado	Art. 37, Art. 47

Descripción:

El archivo `.env` está commiteado en el repositorio git con una API key real y activa de Google Cloud:

```
GOOGLE_API_KEY=AIzaSyAEYHv5NaASuGOhdYULi3tOnqxmUetEvEY

VOICE_SIMULATOR_API_KEY=test_key_123
```

Impacto:

- Acceso no autorizado a servicios de Google Cloud
- Posibles cargos económicos al propietario de la cuenta
- Cualquier persona con acceso al repositorio puede impersonar la aplicación

Remediación inmediata:

1. Revocar la clave en Google Cloud Console **de inmediato**
2. Eliminar del historial de git:

```
  git filter-repo --path backend/.env --invert-paths
```

1. Agregar `.env` al `.gitignore`
2. Usar un gestor de secretos (AWS Secrets Manager, HashiCorp Vault, o variables de entorno del servidor)

CRÍTICO-02: Autenticación débil en WebSocket

Campo	Detalle
Severidad	CRÍTICO
Categoría	Autenticación
Archivo	`backend/src/server.ts:382-391`
Artículo LOPDP relacionado	Art. 37, Art. 39

Descripción:

El API key se transmite como parámetro en la URL del WebSocket (`ws://host?key=test_key_123`), es una cadena estática sin expiración y sin rate limiting.

```
// Código vulnerable actual
const apiKey = url.searchParams.get('key');

const expectedKey = process.env.VOICE_SIMULATOR_API_KEY;

if (!expectedKey || apiKey !== expectedKey) { ... }
```

Impacto:

- La clave es visible en logs de servidor, logs de red y herramientas del navegador
- Sin expiración: una clave comprometida es válida indefinidamente
- Sin rate limiting: ataques de fuerza bruta posibles

Remediación:

- Usar JWT con tiempo de expiración (access tokens de corta duración)
- Mover la autenticación a headers del WebSocket, no a la URL
- Implementar rate limiting en intentos de conexión
- Generar tokens por sesión, no globales

---

2. Vulnerabilidades ALTAS

ALTO-01: Sin autenticación ni autorización en ningún endpoint

Campo	Detalle
Severidad	ALTO
Categoría	Control de acceso
Archivos	`backend/src/server.ts` — todos los endpoints

Descripción:

Todos los endpoints REST son públicos. No existe ningún mecanismo de autenticación. Cualquier persona puede crear, modificar o eliminar campañas, sesiones y números telefónicos.

Impacto: Acceso irrestricto a todos los datos del sistema.

Remediación: Implementar JWT + middleware de autorización por rol.

ALTO-02: CORS demasiado permisivo

Campo	Detalle
Severidad	ALTO
Categoría	Configuración de seguridad
Archivo	`backend/src/server.ts:18-22`

Código vulnerable:

```
app.use(cors({
    origin: '*',  // Permite CUALQUIER origen

    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],

}));
```

Remediación:

```
app.use(cors({
    origin: process.env.ALLOWED_ORIGINS?.split(',') || [],

    methods: ['GET', 'POST'],

}));
```

ALTO-03: Sin validación de entrada en endpoints críticos

Campo	Detalle
Severidad	ALTO
Categoría	Validación de entrada
Archivos	`backend/src/routes/extraction.ts`, `backend/src/server.ts`

Problemas:

- Transcripciones sin límite de tamaño (DoS posible)
- `extractionSchema` no validado antes de enviarse a Gemini
- Números de teléfono validados solo por longitud de dígitos
- IDs de campaña no verificados contra existencia en base de datos

Remediación: Usar Zod o Joi para validar todos los inputs en los endpoints.

ALTO-04: Inyección de prompts no prevenida

Campo	Detalle
Severidad	ALTO
Categoría	Seguridad de IA
Archivo	`backend/src/routes/extraction.ts:22-32`

Código vulnerable:

```
const prompt = `
TRANSCRIPT:

${transcript}          // Input del usuario sin sanitizar


EXTRACTION SCHEMA:

${extractionSchema}    // Input del usuario sin sanitizar

`;
```

Impacto:

- Prompt injection para manipular el comportamiento del LLM
- Extracción de información sensible del contexto del sistema
- Potencial jailbreak del modelo

Remediación: Usar `function calling` / JSON mode de Gemini en lugar de interpolación de strings. Validar y sanitizar inputs antes de incluirlos en prompts.

ALTO-05: Fuga de información en mensajes de error

Campo	Detalle
Severidad	ALTO
Categoría	Manejo de errores
Archivos	`backend/src/routes/extraction.ts:48`, `backend/src/server.ts:370`

Problemas:

- Output raw del LLM retornado al cliente en errores 500
- API key parcialmente logueada en consola (`substring(0,5)`)
- Stack traces que revelan estructura interna

Remediación: Mensajes de error genéricos para el cliente; detalles solo en logs de servidor con enmascaramiento de secretos.

ALTO-06: Mensajes WebSocket sin validación de esquema

Campo	Detalle
Severidad	ALTO
Categoría	Validación de entrada
Archivo	`backend/src/server.ts:232-342`

Código vulnerable:

```
if (msg.type === 'config') {
    agent = createGeminiLiveAgent({

        systemInstruction: msg.systemInstruction,  // Sin validación

        voiceName: msg.voiceName,                  // Sin validación

    });

}
```

Remediación: Validar todos los mensajes WebSocket contra un JSON Schema estricto. Implementar whitelist de tipos de mensajes permitidos. Limitar tamaño de mensajes.

3. Vulnerabilidades MEDIAS

#	Vulnerabilidad	Archivo	Remediación
M-01	Sin rate limiting en ningún endpoint	`server.ts`	Agregar `express-rate-limit`
M-02	API key con prefijo `NEXT_PUBLIC_` (expuesta al cliente)	`frontend/.env.local`	Autenticación server-side
M-03	IDs de sesión generados con `Math.random()` (no criptográfico)	`server.ts`	Usar `crypto.randomUUID()`
M-04	Sin protección CSRF en endpoints de escritura	Todos los endpoints	CSRF tokens + SameSite cookies
M-05	URLs hardcodeadas `http://localhost:3008` en código de producción	`VoiceLiveClient.ts:49`	Solo variables de entorno
M-06	Sin headers de seguridad HTTP (CSP, HSTS, X-Frame-Options)	Frontend Next.js	Configurar en `next.config.ts`
M-07	Sin logging estructurado ni monitoreo de eventos de seguridad	Todo el backend	Winston/Pino + enmascaramiento

---

4. Hallazgos Adicionales

Almacenamiento en memoria (sin persistencia)

Todos los datos (sesiones, campañas, números de teléfono) se almacenan en `Map` de JavaScript en memoria. Se pierden al reiniciar el servidor. No apto para producción.

Remediación: Implementar base de datos (PostgreSQL + Prisma recomendado).

Dependencias — Verificar con npm audit

```
cd backend && npm audit

cd frontend && npm audit
```

Versiones a revisar especialmente: `express@5.2.1`, `socket.io@4.8.3`, `next@16.1.6`

---

5. Plan de Remediación

Fase 1 — Inmediato (0-7 días)

- [ ] Revocar Google API key expuesta
- [ ] Limpiar historial de git del archivo `.env`
- [ ] Agregar `.env` y `.env.local` al `.gitignore`
- [ ] Eliminar `NEXT_PUBLIC_VOICE_SIMULATOR_API_KEY` del frontend

Fase 2 — Corto plazo (1-2 semanas)

- [ ] Implementar JWT para autenticación de usuarios
- [ ] Restringir CORS a orígenes específicos
- [ ] Agregar `express-rate-limit` a todos los endpoints
- [ ] Validar todos los inputs con Zod
- [ ] Mover autenticación WebSocket a headers con tokens JWT

Fase 3 — Mediano plazo (2-4 semanas)

- [ ] Agregar headers de seguridad en `next.config.ts`
- [ ] Reemplazar `Math.random()` por `crypto.randomUUID()`
- [ ] Implementar logging estructurado (Winston/Pino)
- [ ] Implementar base de datos persistente
- [ ] Eliminar URLs hardcodeadas

Fase 4 — Largo plazo (continuo)

- [ ] Pruebas de penetración por tercero
- [ ] Revisiones de seguridad en cada PR
- [ ] Actualización periódica de dependencias
- [ ] Capacitación en seguridad del equipo de desarrollo

6. Tabla Resumen

#	Vulnerabilidad	Severidad	Archivo Principal
1	API Key Google en git	CRÍTICO	`backend/.env`
2	Auth WebSocket débil	CRÍTICO	`server.ts:382`
3	Sin autenticación/autorización	ALTO	`server.ts`
4	CORS permisivo (`*`)	ALTO	`server.ts:18`
5	Sin validación de entrada	ALTO	`extraction.ts`, `server.ts`
6	Inyección de prompts	ALTO	`extraction.ts:22`
7	Fuga de info en errores	ALTO	`extraction.ts:48`
8	Mensajes WS sin validación	ALTO	`server.ts:232`
9	Sin rate limiting	MEDIO	`server.ts`
10	API key en frontend público	MEDIO	`frontend/.env.local`
11	IDs no criptográficos	MEDIO	`server.ts`
12	Sin CSRF	MEDIO	Todos los endpoints
13	URLs hardcodeadas	MEDIO	`VoiceLiveClient.ts`
14	Sin headers HTTP	MEDIO	Frontend
15	Sin logging de seguridad	MEDIO	Backend

Reporte generado el 12 de marzo de 2026 — ELjuri Voice Simulator — Uso interno y confidencial